NextIn

A Virtual Queue App

Suchir Jolapara - U08469805
Namratha Reddy - U65966062
Vijay Krishna - U63589838



NextIn | Virtual Queue Management

WHAT WE ARE SOLVING:

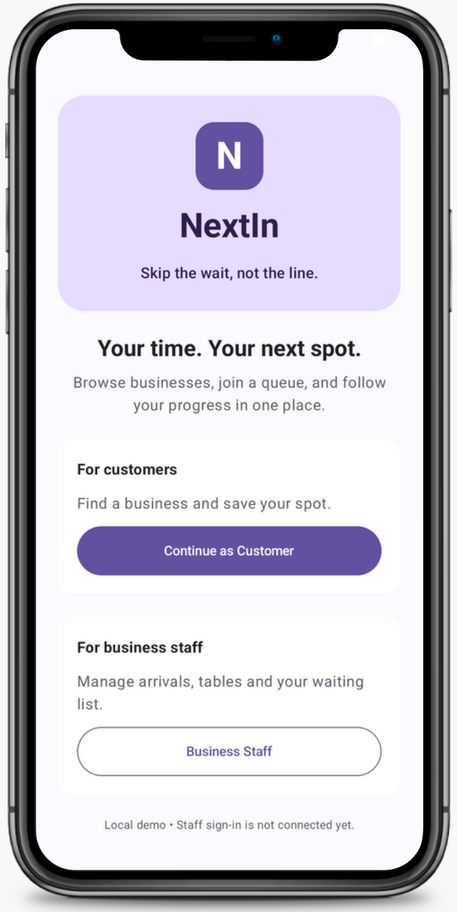
- Walk-in customers often don't know how long they'll wait
- Staff manage waiting lists, walk-ins, and restaurant seating

OUR SOLUTION:

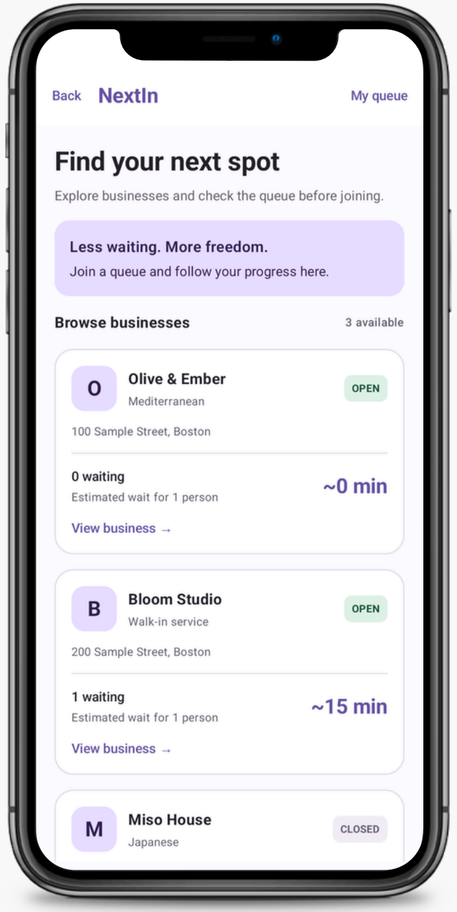
- Customers: Browse businesses, join virtual queues, and track their position and estimated wait
- Staff: Manage waiting parties, add walk-ins, and call or seat customers
- Restaurants: Support FCFS and Table-fit seating based on party size and availability



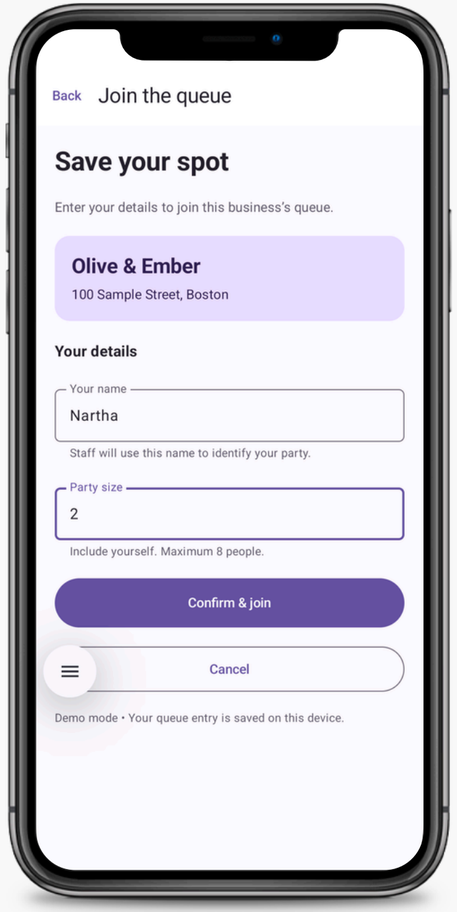
Working Features



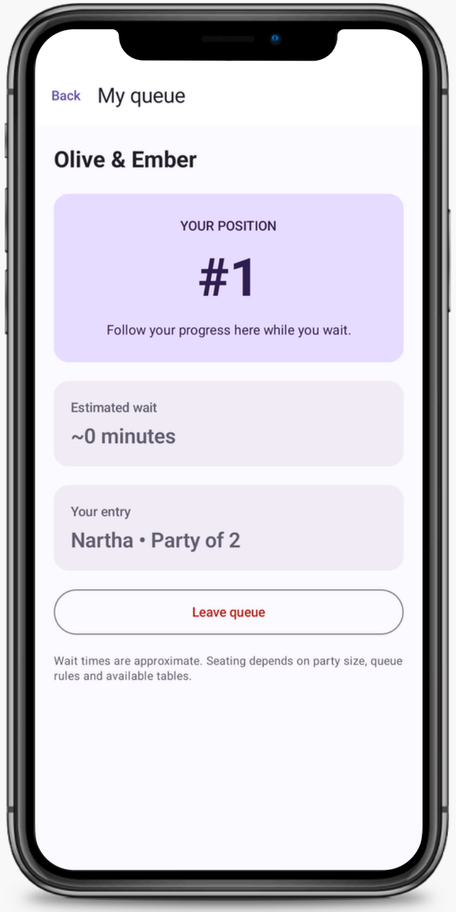
Home Page



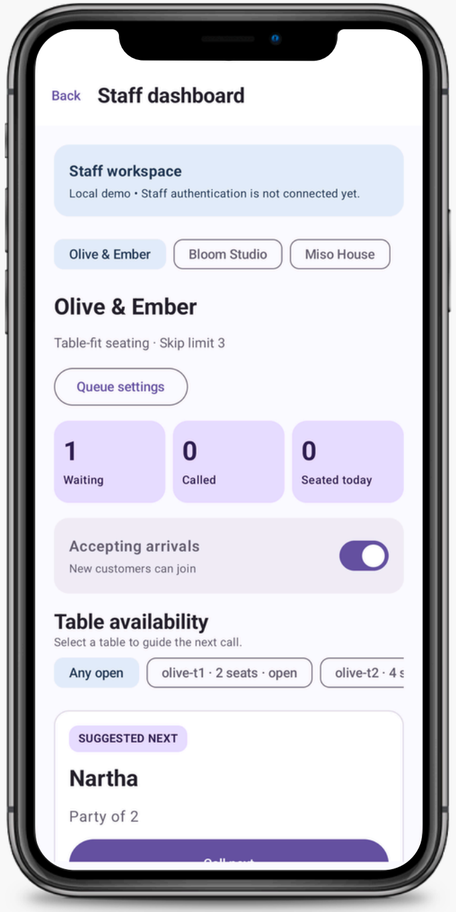
Browsing



Join Queue



Current Queue



Staff View

Live Demo



1. Customer: Welcome → Browse → Business details → Join (name, party size, one active queue per customer)

2. Customer: “My queue” shows position and estimated wait

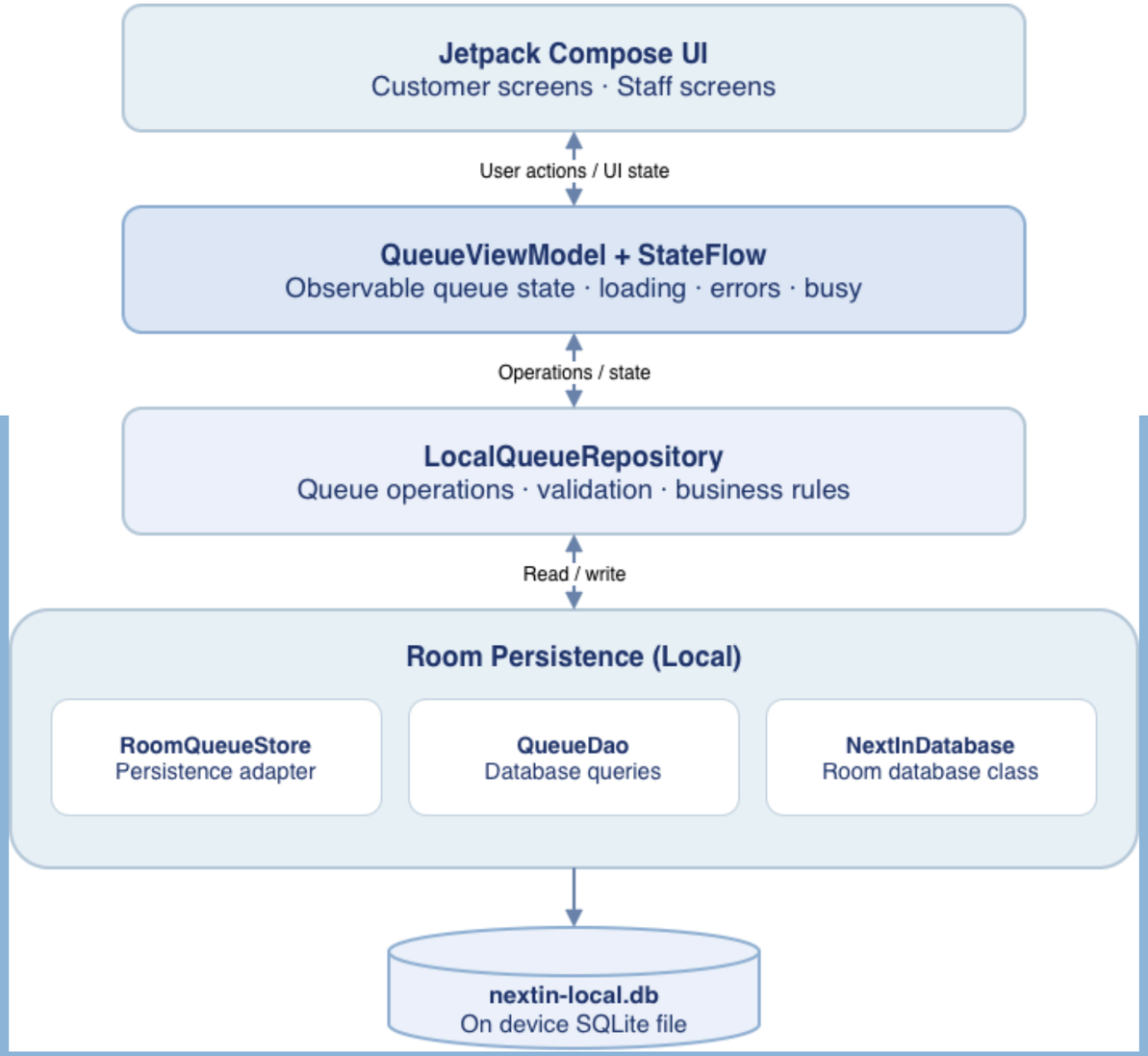
3. Staff: Dashboard → add walk-in → suggested next → call party (grace period starts)

4. Customer: “I’m Here” confirms arrival or if the timer expires, staff see the party expired

5. Restart test: force-close and reopen the app to verify Room persistence

Architecture and Persistence

- Jetpack Compose: Displays queue information and handles user actions
- QueueViewModel + StateFlow: Manages observable queue state, loading, and errors
- LocalQueueRepository: Processes queue operations and business rules
- Room Persistence: RoomQueueStore, QueueDao, and NextInDatabase save and load queue data, including sample businesses



Backlog Progress and Scope

Currently Implemented:
Android architecture and navigation, queue models, customer browse/join flow, local queue operations and Room integration

Implemented and Verified:
Staff workflows, queue status, wait estimates, grace period behavior and room persistence across app restarts

Remaining:
Firebase shared queues and authentication, Google Places, location, notifications, Wear OS, additional UI tests and accessibility

Scope change:
Update 1 uses Room as the local data source behind the repository. Firebase shared queues and auth move to Update 2

Status

Priority	Feature	Status
P0	Verify local queue and Room persistence	Implemented
P0	Firebase shared queues and authentication	Planned
P0	Google Places and recommendations	Planned
P1	Location and permissions	Planned
P1	Notifications and Wear OS	Planned
P1	UI testing, accessibility, robustness	Planned

Risks and Mitigation

Type	Risk	Mitigation
Schedule	Firebase, Places, location, notifications and Wear OS all land before Nov 24	Cut order: watch refresh, extra filters, simpler Table-fit settings
Concurrency	Local demo is one device with one demo customer ID, Firestore transactions untested	Firestore transactions, unit tests, emulator
Clock	Grace period countdown uses device time	Use Firestore server calledAt timestamp
Privacy/ Permission	Names and location are sensitive	No phone numbers or location history, location and notifications optional
External API	Places quotas, cost, key exposure	Restrict key, cache in Room, keep keys out of Git
Device	Wear OS phone to watch sync is hard	Start as a read-only status screen

GitHub Repository and Contribution

Namratha:

- Initial architecture and navigation
- Project proposal and planning
- Integration/documentation/testing
- Live customer demonstration

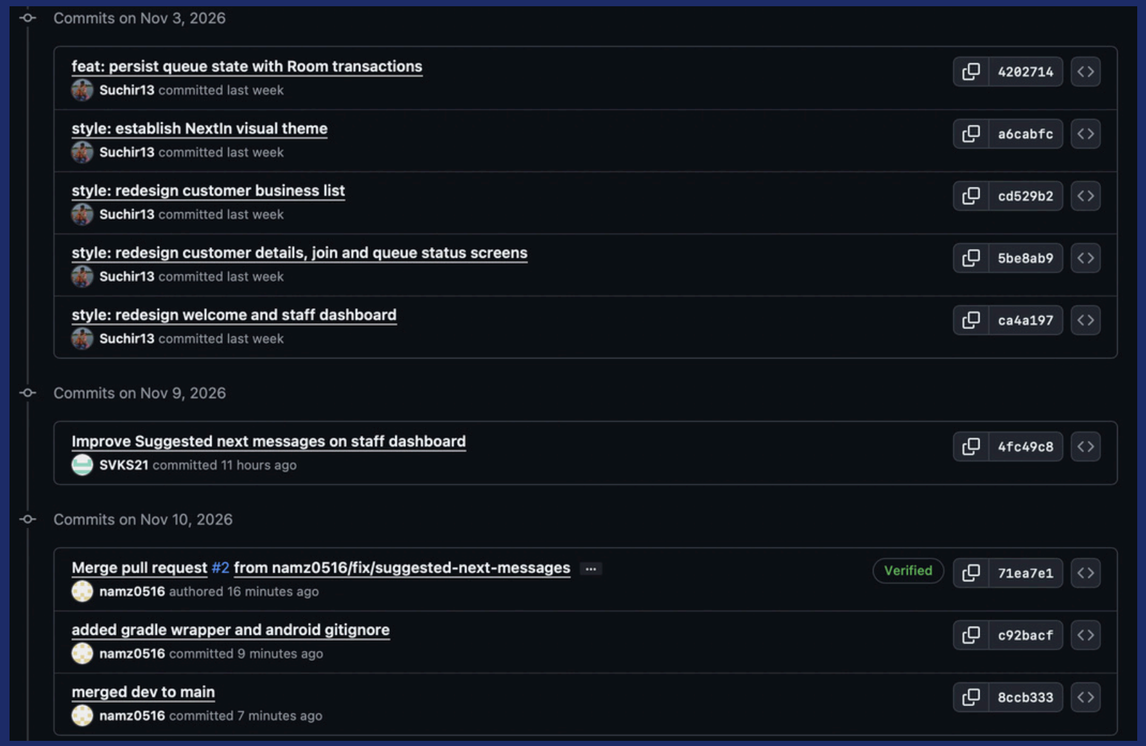
Suchir (Scrum Master from Nov 10-16):

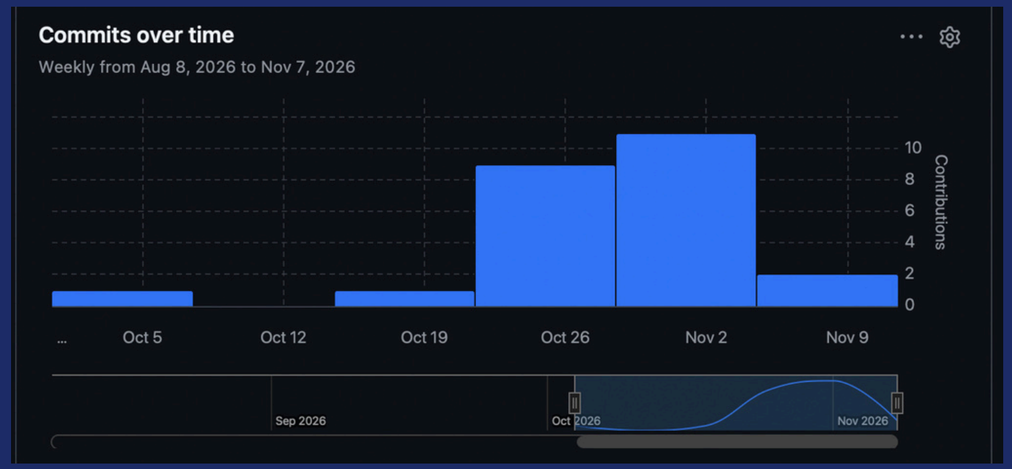
- Queue models and business logic
- Customer and staff implementation
- Room persistence and unit tests
- Live staff/persistence demonstration

Vijay:

- QA testing of customer and staff flows
- Bug reports and suggested improvements

Git URL: https://github.com/namz0516/CS501-NextIn





Next Steps and Team Plan

- Firebase Auth + Firestore shared queues (Suchir)
- Google Places recommendations, optional location, DataStore (Namratha)
- Notifications and grace period across devices (Vijay)
- Compose UI test for the join flow, security rules tested on the Firebase emulator (all)
- Workflow: branch → PR → review → merge
- Scrum master rotates weekly

Next Milestone: Nov 24 for Project Update 2

Goal: Feature complete with working major integrations, early automated tests, and documented defects

AI Disclosure & References

1. Android Developers. "Jetpack Compose." Android Developers.
2. Android Developers. "Navigation with Compose." Android Developers.
3. Android Developers. "ViewModel," "StateFlow," and "Kotlin coroutines on Android." Android Developers.
4. Android Developers. "Room" and "DataStore." Android Developers.
5. Android Developers. "Request location permissions" and "Fused location provider." Android Developers.
6. Android Developers. "Wear OS" and "Compose for Wear OS," "Data Layer API." Android Developers.
7. Google Firebase. "Cloud Firestore." Firebase Documentation.
8. Google Firebase. "Firebase Authentication." Firebase Documentation.
9. Google Firebase. "Firestore Security Rules." Firebase Documentation.
10. Google Maps Platform. "Places API." Google for Developers.

ChatGPT was used to brainstorm project ideas and app names, discuss limitations to help define scope, review the project requirements, and assess technical decisions and risks. Figma was used to design the early UI screens

We reviewed every scope and technical decision

THANK YOU

Questions/ Feedback



